

## **I.4**

# **La couche Réseau**

## **I.4. La couche Réseau**

---

**I.4.1. Définitions**

**I.4.2. Le routage**

**I.4.3. Le contrôle de flux**

**I.4.4. Services de la couche réseau**

**I.4.5. La norme X25, niveau réseau**

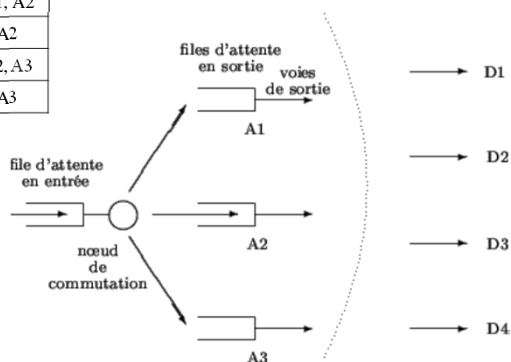
---

- Acheminement des données de bout en bout (traverser plusieurs nœuds)
- Trouver le bon chemin entre un émetteur et un récepteur : routage
- Autres fonctionnalités
  - Contrôle de flux et de congestion
  - Gestion des erreurs
  - Segmentation et réassemblage
- **Paquet** : unité d'information au niveau réseau, taille variable (entête, information)
- Router le paquets nécessite un plan d'adressage pour le réseau
- **Adresse réseau** : identifie de manière unique un nœud sur le réseau

- Le **routage** des paquets consiste à fixer par quelle ligne de sortie chaque routeur réexpédie les paquets qu'il reçoit
- Le routage se fait :
  - En fonction de la destination finale
  - Selon une table de routage qui indique pour chaque destination finale quelles sont les voies de sortie possible

❁ Exemple :

| destination finale | voie de sortie |
|--------------------|----------------|
| D1                 | A1, A2         |
| D2                 | A2             |
| D3                 | A2, A3         |
| D4                 | A3             |



❁ Politiques de routage :

❖ Non adaptatif / Adaptatif

- **Non adaptatif** (Déterministe) : tous les paquets suivent le même chemin (commutation)
- **Adaptatif** : décision prise pour chaque paquet en fonction de l'état du réseau

❖ Statique / dynamique

- **Statique** : routes préalablement définies par l'administrateur
- **Dynamique** : routes peuvent changer en fonction de l'état du réseau sans intervention de l'administrateur

• **Algorithmes de routage** (avec connaissance de la topologie du réseau)

❖ **Vecteur de distance**

- Chaque nœud connaît ses adjacents → construit une table de routage
- Échange périodique des tables de routage entre les différents nœuds (trafic supplémentaire, cohérence des tables)

❖ **État de lien**

- Chaque nœud dispose de la vue globale du réseau (graphe)
- Échange d'information de routage après changement de topologie
- Association de coût pour chaque lien entre nœuds adjacents
- Algorithme du chemin le plus court entre un nœud source et un nœud destination (Dijkstra)

• **Algorithmes du chemin à coût minimum :**

❖ **Algorithme de DIJKSTRA**

- trouver le chemin, à coût minimal, d'un nœud source à chacun des autres nœuds du réseau
  - $N$  = Ensemble des nœuds
  - $S$  = nœud source
  - $M$  = contient à tout moment l'ensemble de nœuds dont la distance minimale de la source est connue
  - $L(i,j)$  = coût de lien du nœud  $i$  à  $j$  (= l'infini si l'arc  $(i, j)$  n'existe pas)
  - $C(n)$  = coût du chemin à coût minimal de  $S$  à  $n$

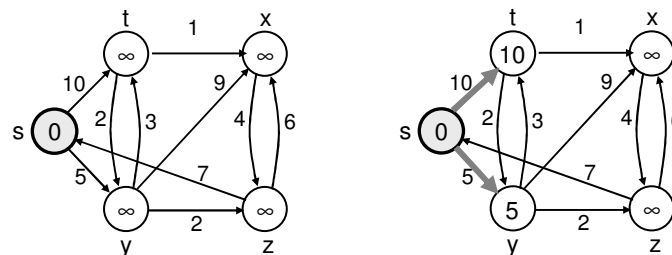
• Algorithmes du chemin à coût minimum :

❖ Algorithme de DIJKSTRA

1.  $M = \{S\}$  (M contient la source)
2. **Pour chaque** noeud  $n$  de  $N - \{S\}$  **faire**  $C(n) = L(s, n)$
3. Trouver  $W$  de  $N - M$  tel que  $C(W)$  est minimum et l'ajouter à  $M$  ( $M$  devient  $M + \{W\}$ )
4. **Pour chaque** noeud  $n$  de  $N - M$  **faire**  $C(n) = \text{Min. } [C(n), C(W) + L(W, n)]$
5. Répéter (3 et 4) jusqu'à ce que  $M = N$

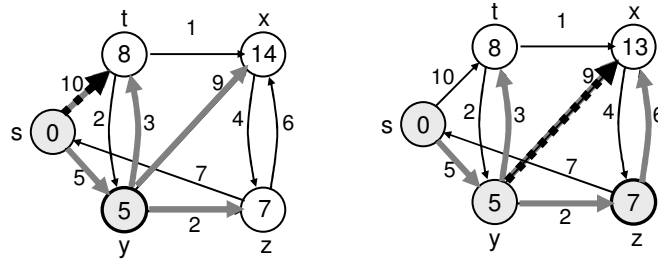
• Algorithmes du chemin à coût minimum :

❖ Algorithme de DIJKSTRA (Exemple 1)



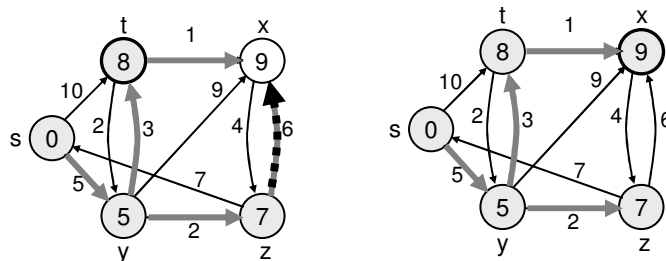
• Algorithmes du chemin à coût minimum :

❖ Algorithme de DIJKSTRA (Exemple 1)



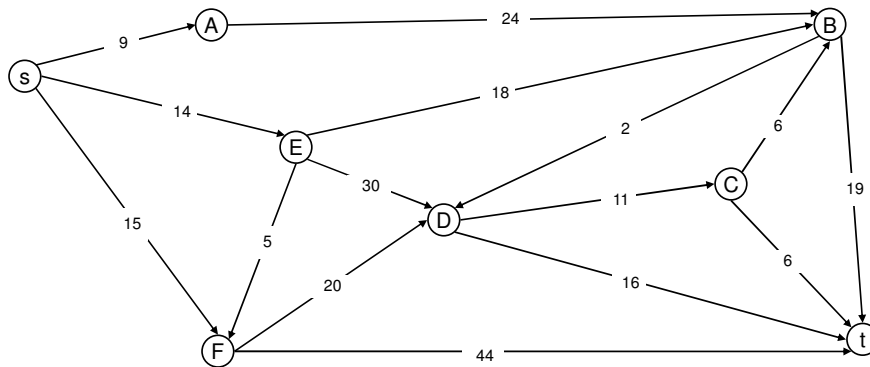
• Algorithmes du chemin à coût minimum :

❖ Algorithme de DIJKSTRA (Exemple 1)



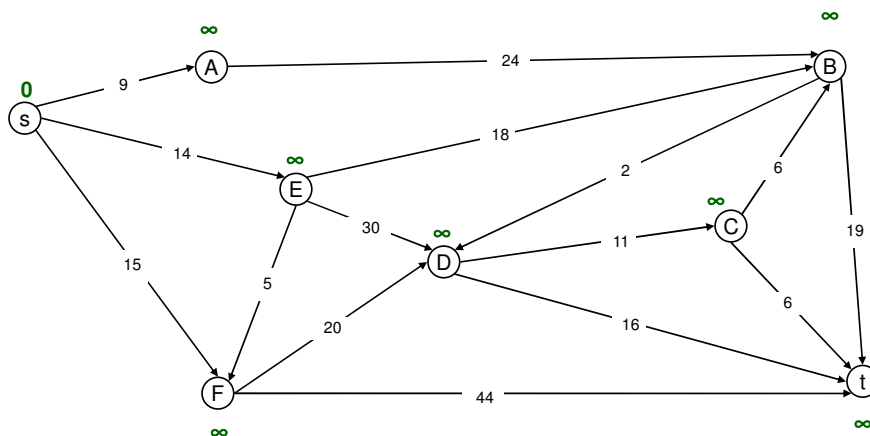
❁ Algorithmes du chemin à coût minimum :

❖ Algorithme de DIJKSTRA (Exemple 2)



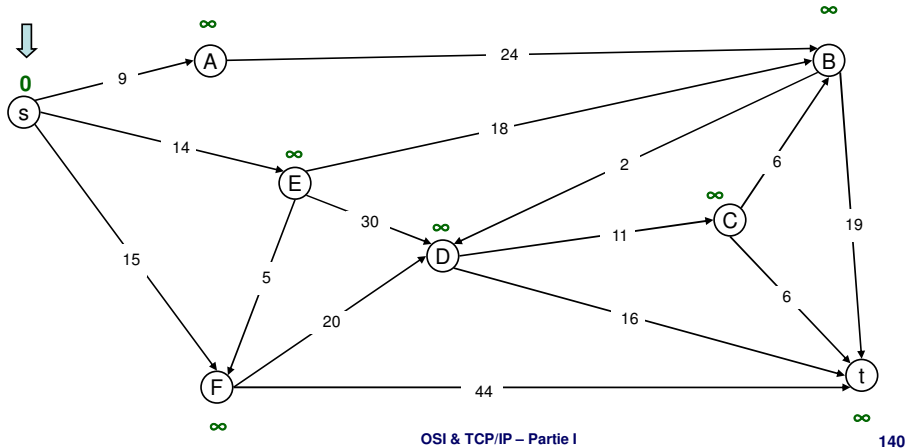
❁ Algorithmes du chemin à coût minimum :

❖ Algorithme de DIJKSTRA (Exemple 2)



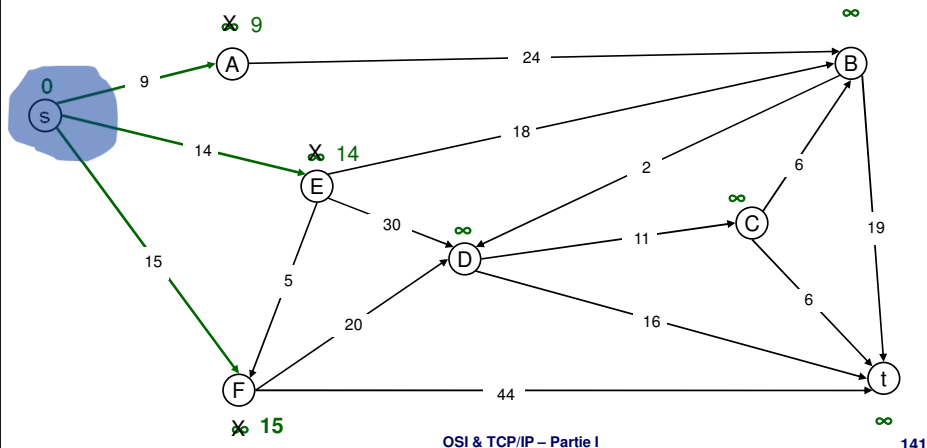
Algorithmes du chemin à coût minimum :

❖ Algorithme de DIJKSTRA (Exemple 2)



Algorithmes du chemin à coût minimum :

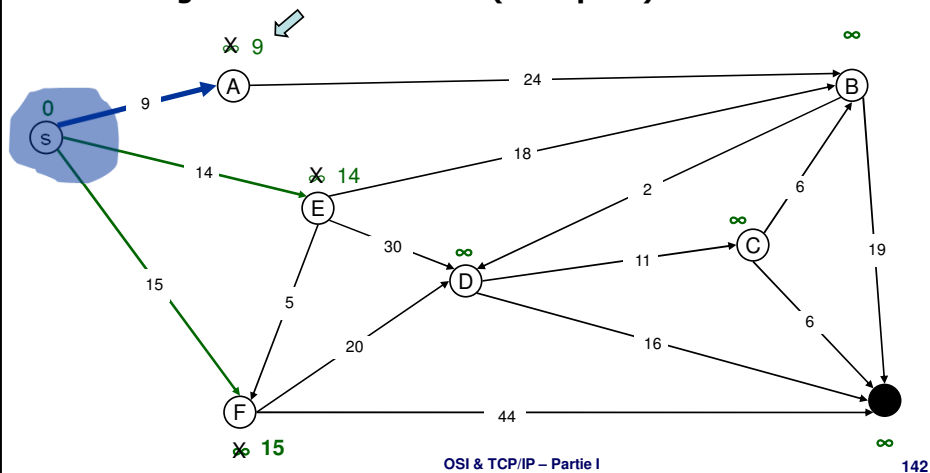
❖ Algorithme de DIJKSTRA (Exemple 2)





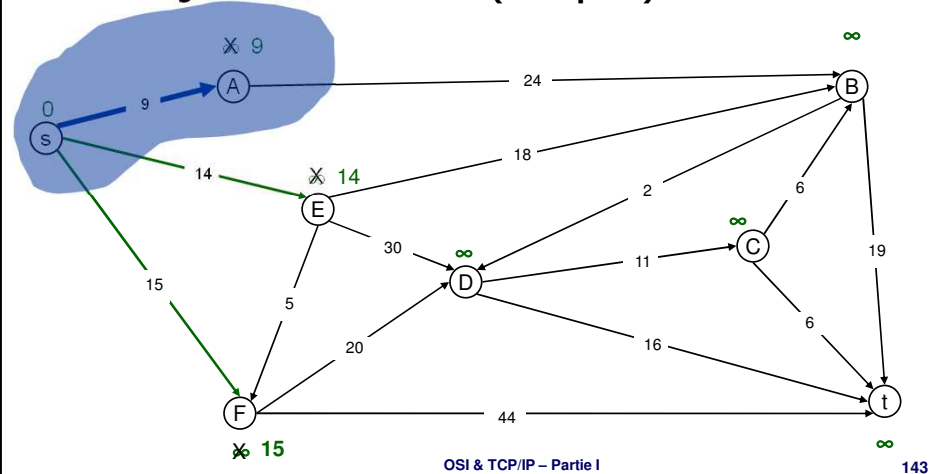
Algorithmes du chemin à coût minimum :

❖ Algorithme de DIJKSTRA (Exemple 2)



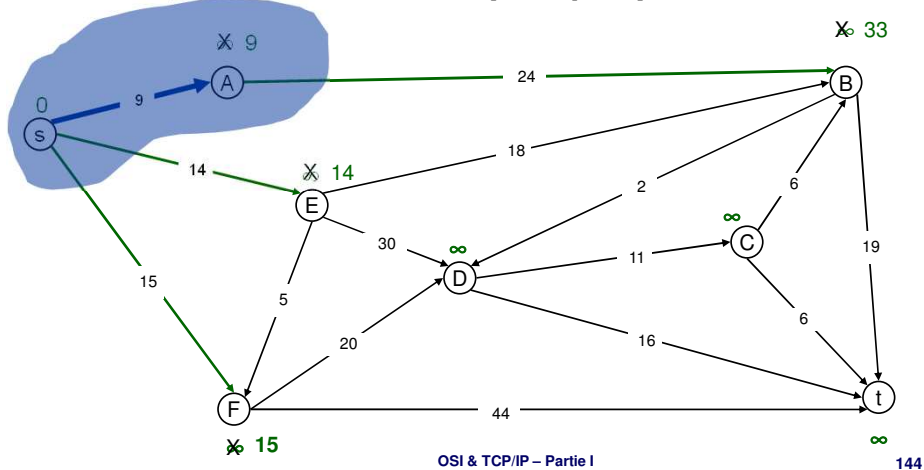
Algorithmes du chemin à coût minimum :

❖ Algorithme de DIJKSTRA (Exemple 2)



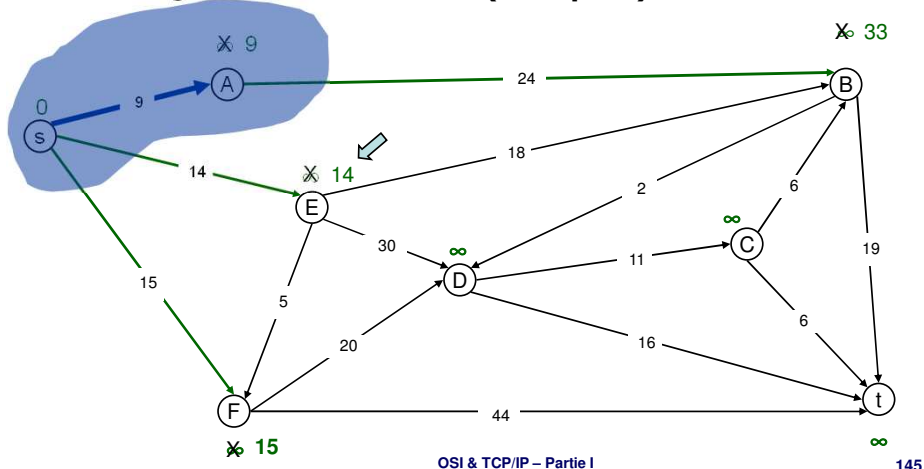
Algorithmes du chemin à coût minimum :

❖ Algorithme de DIJKSTRA (Exemple 2)



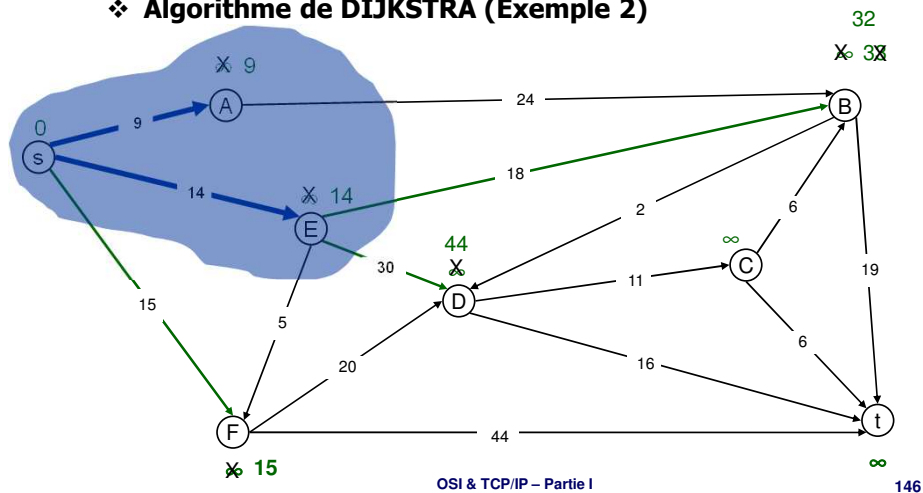
Algorithmes du chemin à coût minimum :

❖ Algorithme de DIJKSTRA (Exemple 2)



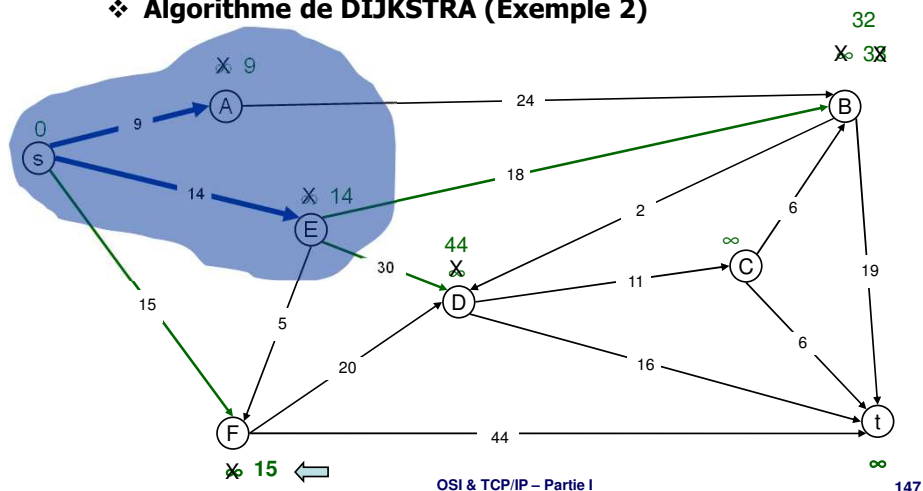
Algorithmes du chemin à coût minimum :

❖ Algorithme de DIJKSTRA (Exemple 2)



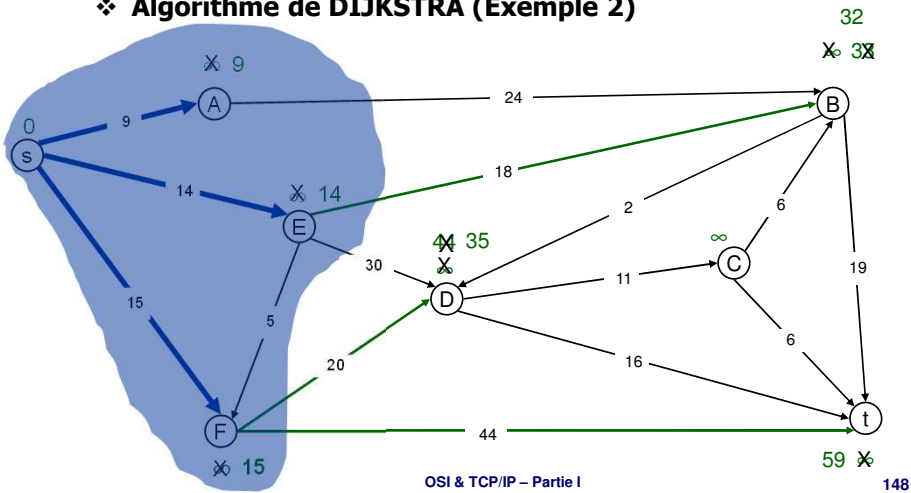
Algorithmes du chemin à coût minimum :

❖ Algorithme de DIJKSTRA (Exemple 2)



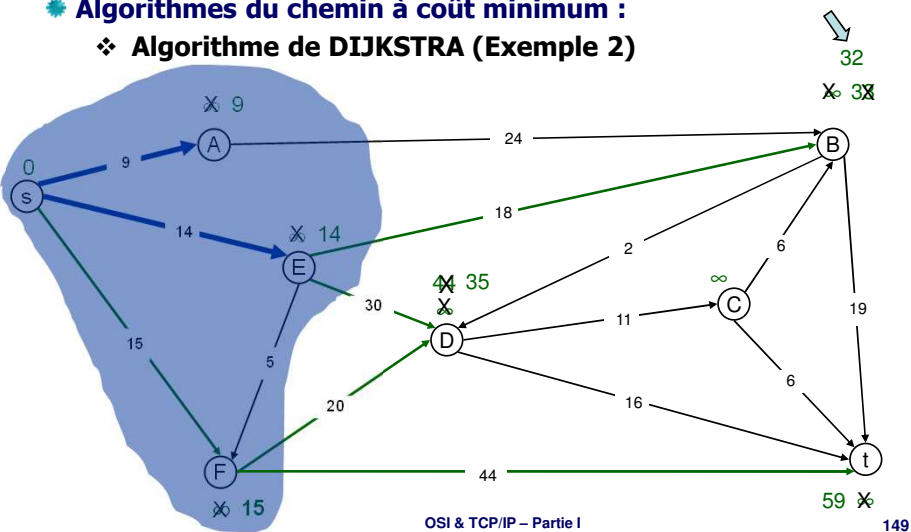
Algorithmes du chemin à coût minimum :

❖ Algorithme de DIJKSTRA (Exemple 2)



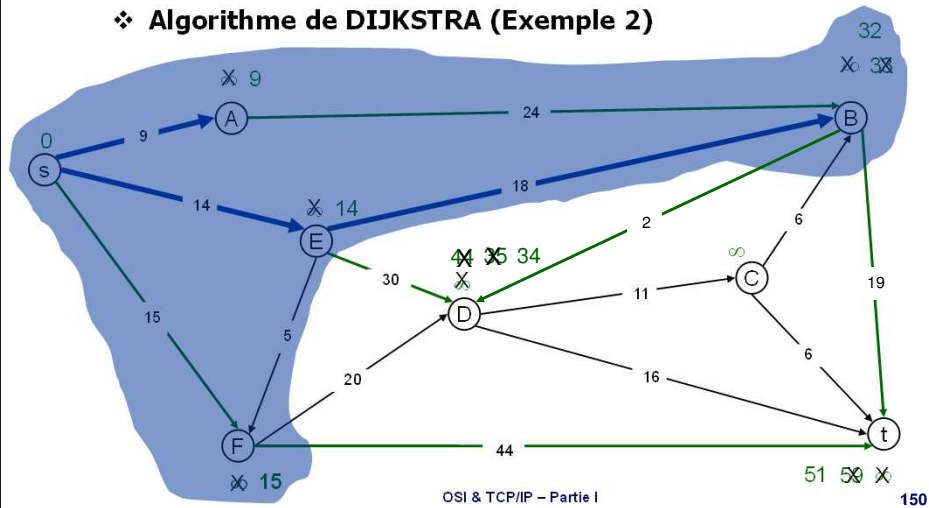
Algorithmes du chemin à coût minimum :

❖ Algorithme de DIJKSTRA (Exemple 2)



❁ Algorithmes du chemin à coût minimum :

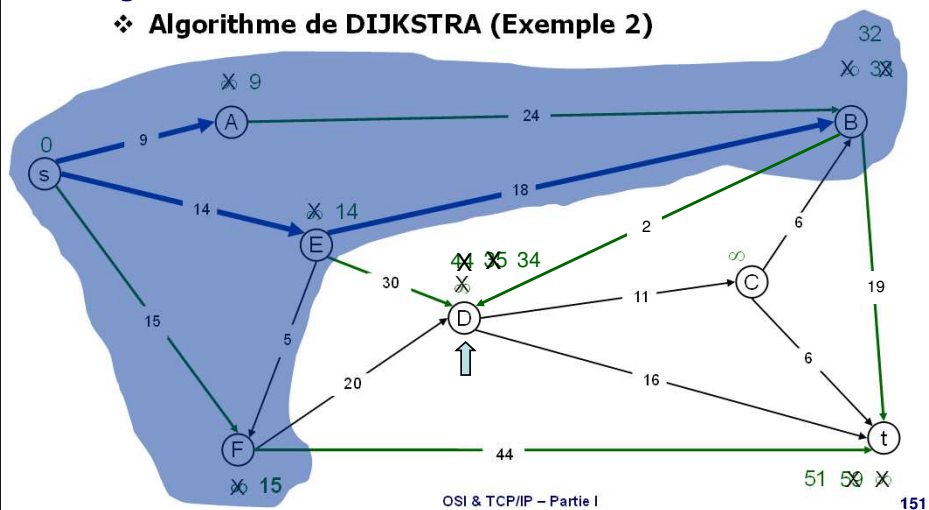
❖ Algorithme de DIJKSTRA (Exemple 2)



150

❁ Algorithmes du chemin à coût minimum :

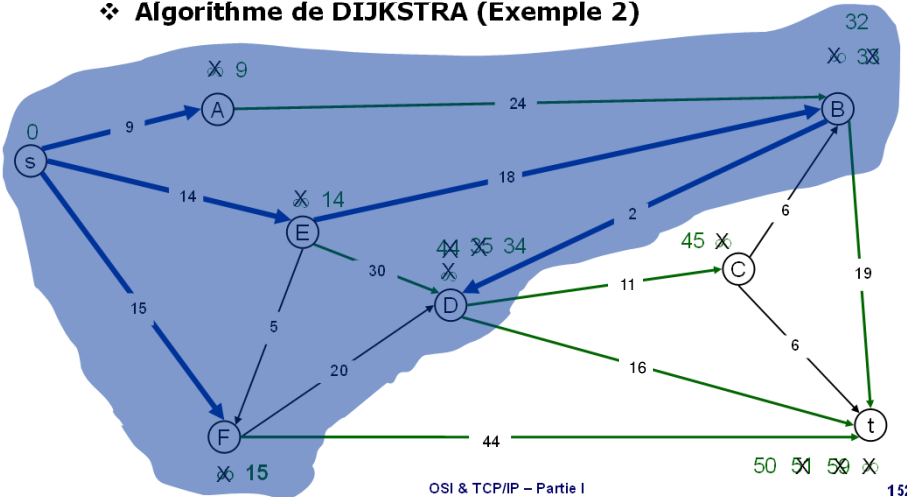
❖ Algorithme de DIJKSTRA (Exemple 2)



151

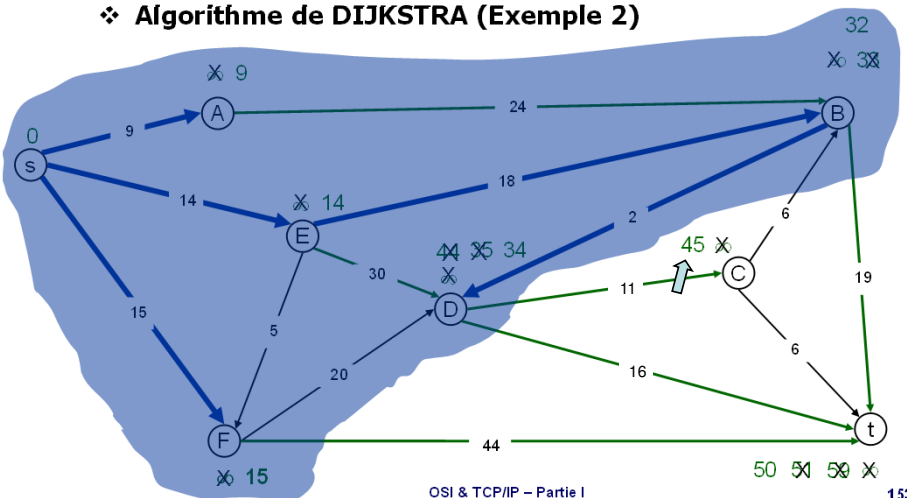
Algorithmes du chemin à coût minimum :

❖ Algorithme de DIJKSTRA (Exemple 2)



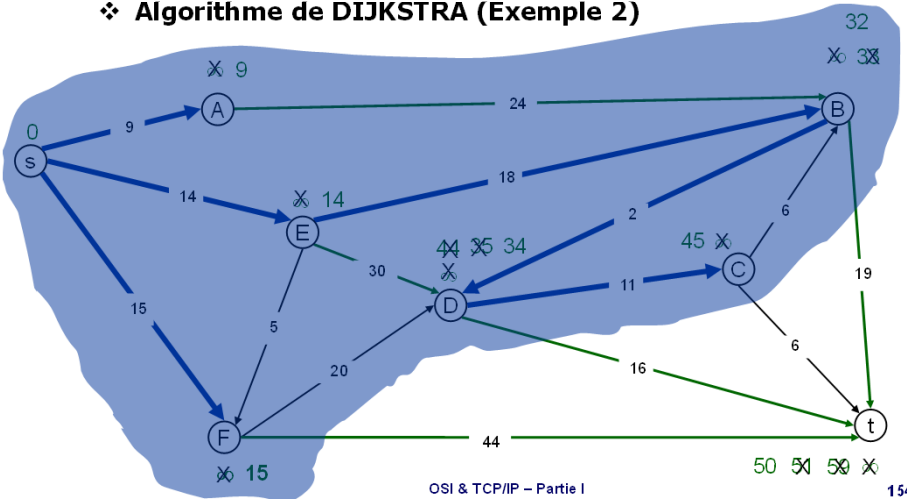
Algorithmes du chemin à coût minimum :

❖ Algorithme de DIJKSTRA (Exemple 2)



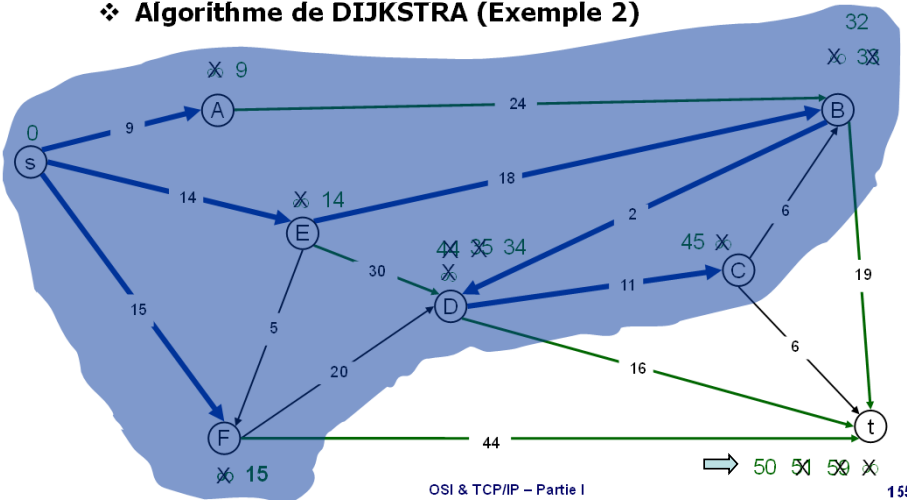
✿ Algorithmes du chemin à coût minimum :

❖ Algorithme de DIJKSTRA (Exemple 2)



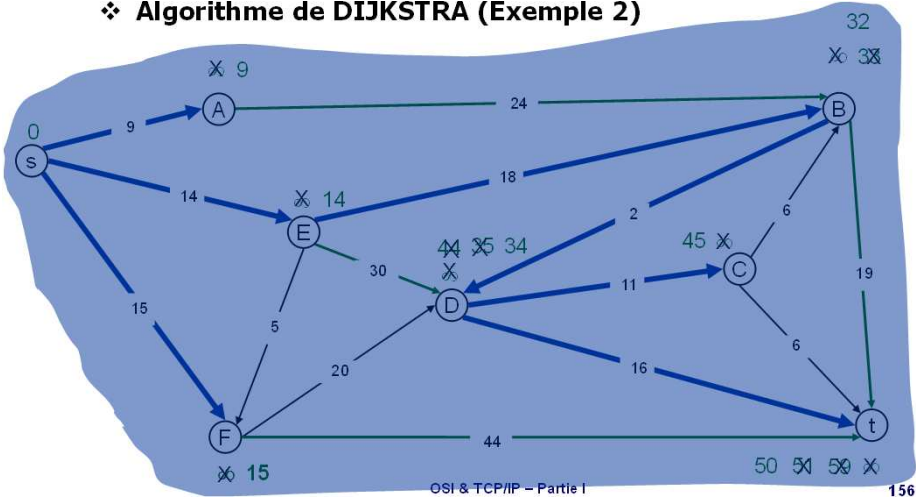
✿ Algorithmes du chemin à coût minimum :

❖ Algorithme de DIJKSTRA (Exemple 2)



✿ Algorithmes du chemin à coût minimum :

❖ Algorithme de DIJKSTRA (Exemple 2)



✿ Algorithmes du chemin à coût minimum :

❖ Algorithme de FORD

- Étant donné une destination D, l'algorithme de FORD trouve le chemin, à coût minimum, de n'importe quel autre noeud pour se rendre à D.



• **Algorithmes de routage Non adaptatif :**

❖ **Routage fixe :**

- Tous les paquets entrés en  $N_E$  à destination de  $N_D$  emprunteront toujours le même chemin
- Les tables d'acheminement sont fixes
- Très simple à mettre en oeuvre
- Peu sûr

❖ **Routage par inondation :**

- Un noeud envoie le paquet à tous les noeuds voisins et ainsi de suite de proche en proche jusqu'au destinataire
- Simple et très sûr
- Risque de congestion

• **Algorithmes de routage adaptatif :**

❖ **Algorithme isolé :**

- Chaque noeud applique un algorithme ne tenant compte que des **informations locales** pour tenir à jour sa table d'acheminement
- **Exemples :**
  - hot patats : paquet met dans la queue la plus courte
  - Backword Learning

• **Algorithmes de routage adaptatif :**

❖ **Algorithme centralisé :**

- Nœud qui centralise l'information sur l'état du réseau (RCC = Centre de Contrôle de Routage). Diffusion périodique vers les autres nœuds
- **Avantages :**
  - Décharge les différents nœuds du calcul des tables
  - Le RCC possède l'information complète sur le réseau
- **Inconvénients :**
  - Congestion des lignes qui mènent au RCC
  - Si le RCC tombe en panne, tout le réseau est infecté
  - Si le réseau est grand, la collecte d'informations constitue une grande charge

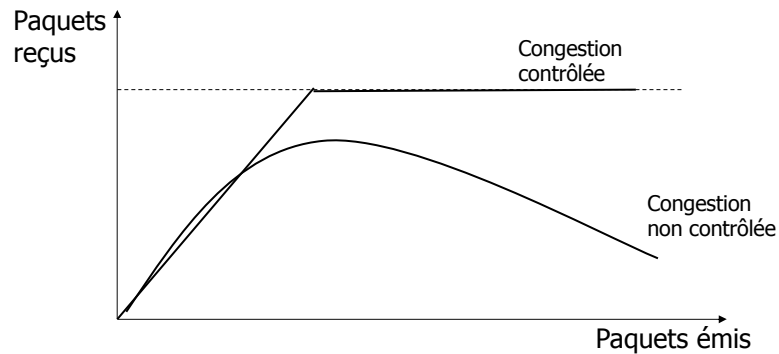
• **Algorithmes de routage adaptatif :**

❖ **Algorithme distribué :**

- Collaboration de plusieurs nœuds pour la gestion du réseau
- Chaque nœud maintient une table de routage qui contient une entrée pour chaque nœud du réseau. Cette entrée contient :
  - L'identification de la ligne à utiliser de préférence pour aboutir à destination
  - Une estimation "temps ou distance" pour atteindre la destination

❁ **Problème de congestion :**

- Congestion du réseau : trop de paquets présents dans le réseau  
→ dégradation des performances



❁ **Contrôle de flux :**

- **Contrôle par crédits :** seuls  $N$  paquets sont autorisés à circuler simultanément sur le réseau, un paquet ne peut entrer dans le réseau qu'après avoir acquis un jeton qu'il relâche lorsqu'il arrive à destination
- **mécanisme de fenêtre :**  $P(S)$  un compteur de paquets émis,  $P(R)$  un compteur de paquets reçus et  $W$  est la taille de la fenêtre d'émission. L'émetteur n'est autorisé à émettre que les paquets inclus dans la fenêtre

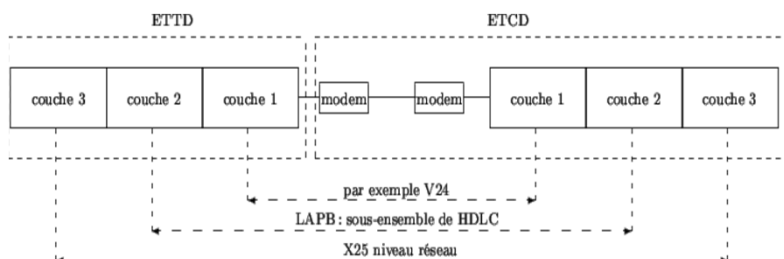
$$\text{dernier } P(R)_{\text{reçu}} \leq P(S)_{\text{courant}} \leq \text{dernier } P(R)_{\text{reçu}} + W$$

- Suppression de paquets qui restent longtemps dans les nœuds

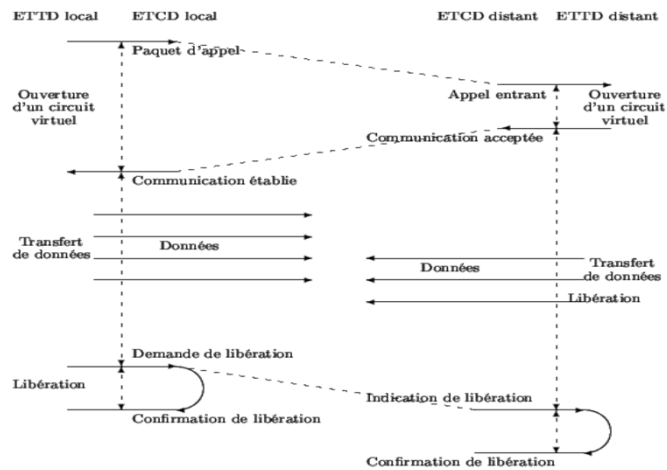
- **Mode non connecté ou datagramme (connexion less) :**
  - Paquets sont transmis **indépendamment** les uns des autres au sein du réseau.
  - Routage est effectué grâce à l'**adresse de destination** (entête du paquet)
  - Routage des paquets adaptatif
  - Exemple : IP
- **Mode connecté :**
  - 3 phases : établissement de connexion, transfert de données et la libération de connexion
  - Exemple : X25

- **Circuit virtuel (mode connecté) :**
  - Paquets acheminés **séquentiellement** grâce à une connexion ouverte au sein du réseau
  - Routage (commutation) effectué grâce à un **identificateur de circuit virtuel** figurant (entête du paquet)
  - La phase d'établissement de la connexion permet de déterminer le circuit virtuel en réservant les ressources (notamment l'identificateur de circuit virtuel) au sein de chaque nœud
  - Tous les paquets utilisant un circuit virtuel suivent le **même chemin**
  - Les paquets sont transmis sur ce circuit virtuel dans l'**ordre**
  - Généralement, un **contrôle de flux et d'erreur** y sont associés

- Établie en **1976** par le **CCITT**



- X25 utilise le mode avec connexion. La connexion est réalisée à l'aide d'un circuit virtuel
- Deux types de connexion possibles :
  - ❖ circuit virtuel permanent (**PVC** : "Permanent Virtual Circuit") : connexion est établie en permanence
    - Fonctionne comme une ligne spécialisée (louée/réservée)
  - ❖ circuit virtuel (**VC** Virtual Circuit) :
    - En trois phases : établissement de la connexion, transfert de données, libération de la connexion



### Format général des paquets X25

#### Paquets de contrôle

|                                                                               |   |   |   |      |
|-------------------------------------------------------------------------------|---|---|---|------|
| 0                                                                             | 0 | 0 | 1 | NGVL |
| NVL                                                                           |   |   |   |      |
| type                                                                          |   |   |   | 1    |
| Données additionnelles<br>adresses source et destination<br>paramètres de QOS |   |   |   |      |

#### Paquets de données

|         |   |      |      |  |
|---------|---|------|------|--|
| D       | Q | Mod  | NGVL |  |
| NVL     |   |      |      |  |
| P(R)    | M | P(S) | 0    |  |
| Données |   |      |      |  |

#### Format des paquets X25

- **NGVL** : numéro de groupe de voies logiques (4 bits=16)
- **NVL** : numéro de voie logique (8 bits=256 )
- 4096 voies logiques différentes

#### Paquets de contrôle

| type de paquets           | valeur   |
|---------------------------|----------|
| ▪ CALL REQUEST (APP)      | 0000101  |
| ▪ CALL ACCEPTED (COM)     | 0000111  |
| ▪ RECEIVE READY (RR)      | P(R)0000 |
| ▪ RECEIVE NOT READY (RNR) | P(R)0010 |
| ▪ REJECT (REJ)            | P(R)0100 |
| ▪ CLEAR REQUEST (LIB)     | 0001001  |
| ▪ CLEAR CONFIRM (CONF)    | 0001011  |
| ▪ INTERRUPT               | 0010001  |
| ▪ INTERRUPT CONFIRM       | 0010011  |
| ▪ .....                   |          |

#### Paquets de données

- Le bit **D** ("Delivery"):
  - 1 : contrôle de flux de bout en bout
  - 0 : contrôle de flux local
- Le bit **Q** ("Qualified data") :
  - 1 : données de contrôle
  - 0 : données issues des entités des couches supérieures
- Le champ **Modulo** :
  - dimension des champs de numérotation : P( S) et P( R),
  - 01 : modulo 8 (entête de 3 octets) - format normal de paquet
  - 11 : modulo 128 (entête de 4 octets) - format étendu de paquet

### • **Paquets de données**

- Les champs **P( S )** et **P( R )** :
  - P( S ) : numéro du paquet
  - P( R ) : numéro du prochain paquet attendu et acquittement des paquets antérieurs (  $P(S) < P(R)$  )
- Le bit **M** (" More"):
  - 1 : ce paquet est le dernier fragment
  - 0 : ce paquet est un fragment intermédiaire